TD N-Lateration, Static-Fingerprint, and Mobility Prediction with HMM

By Philippe Canalda

Introduction

These slots of 2 hours of directed works provide the opportunity to co-conceive and to implement various basic software components helpfull for modern applications wherin positioning and dynamic mobility are crucial.

Summary

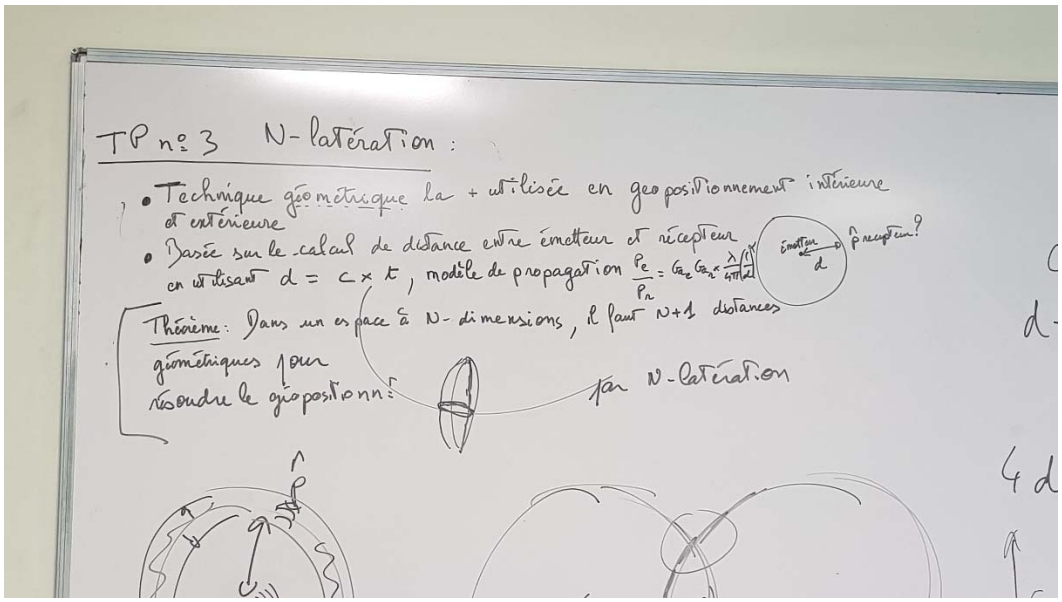
Table des matières

TD n° 2 N-Lateration

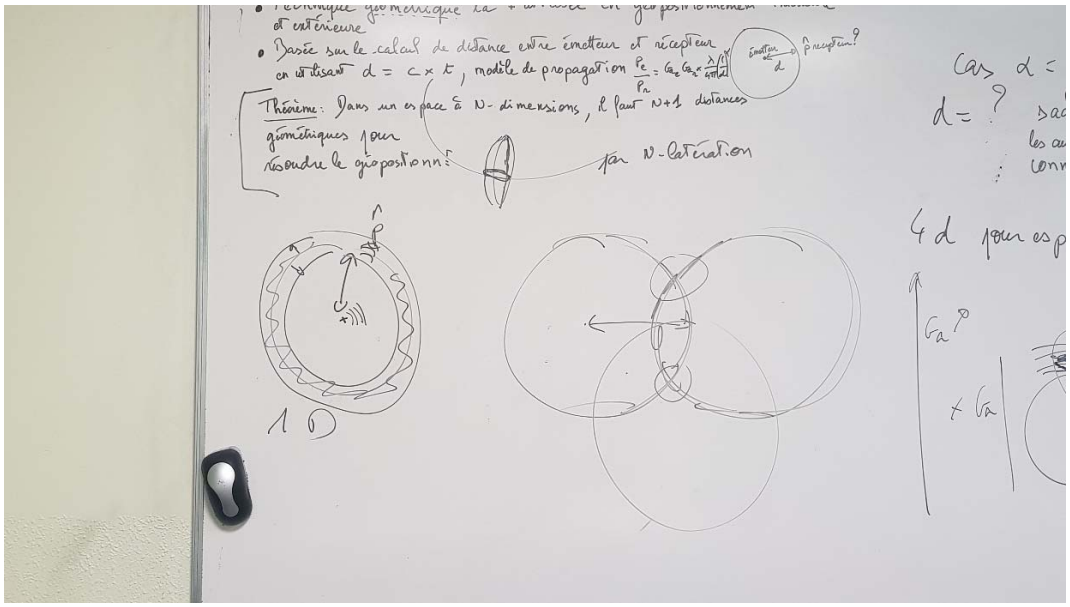
The objective is to conceive and implement, with the language of your choice, the basic N-lateration algorithm. In this directed work, I will propose a global architecture view of an indoor positioning scenario involving N=4 Access Points and an object/human identified as a Mobile Object (Terminal). 2 approaches based, either on model propagation, or Time of Fligth, provide distances between Emitters and Receivers.

N-lateration requires N Emitters and 1 Receiver or 1E-NR.

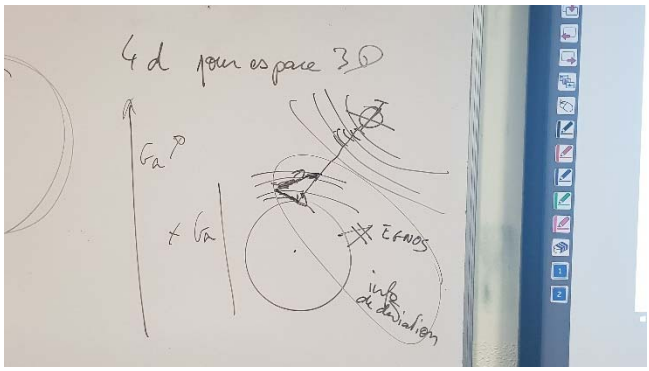
To geolocalize, or to position in a N dimension environment, that requires, at least N+1 distances between Emitter and Receiver which are not co-linear.



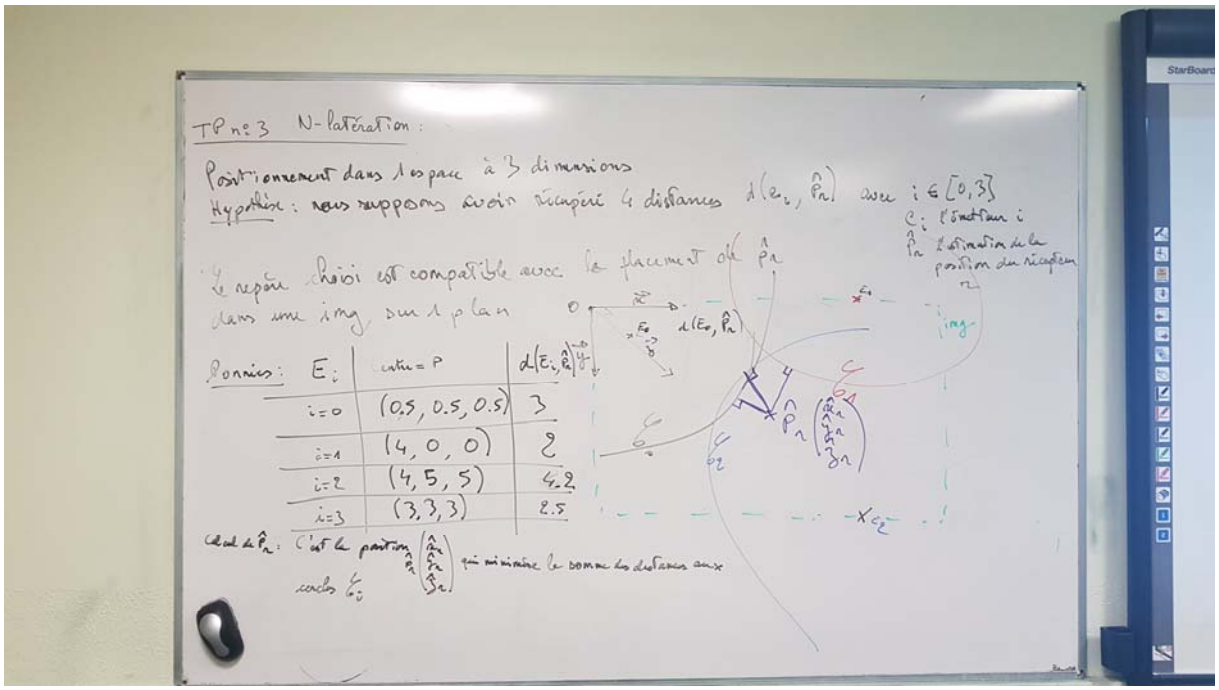
In a corridor (2D), at least 2 distances are required. In a plan, 3 circles. In a 3D space, 4 satellites and a GPS receiver or 1 Emitter and 4 Receivers in infrastructure-centric positioning system.



But, take care that knowing d(E,R) is not sufficient when line of sight are not guarented or when speed-up happen (due to heterogeneity of densities, due to electro-magnetic strength, etc.).



See below the indoor environment to be considered. We need a picture of the environment and a cartesian referential where the origine is placed in the up and left corner, the X axis is right oriented and Y axis is down oriented. We provide a data set composed of 4 distances between Ei emitters and a receiver. The position of emitters are known.



The work to do is :

1. Design the V1 version of the implementation (following scrum/agile methodology), which favorize the time to validation of positioning technic implementation ;
2. Propose the Object Class Diagram with prioritized functions ;
3. Conceive the two main algorithms :
    a. The N-lateration algorithm which compute the estimated position by minimizing the sum of distances to 4-spheres inside the perimeter of the 4 spheric volumes agregated ;
    b. The factory design pattern (for the given data set) ;
4. Compare the result provided by your implementation with a geometric resolution of the senario provided.

As Annexe 1, 5 pictures which illustrate the resolution of this problem.

Below some information to better understand the overall problem.

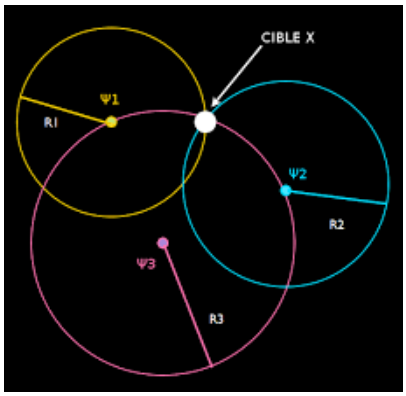
$d = c \times t$ the celerity model

$$\frac{Pe}{Pa} = \frac{G_{ae} \times G_{ar} \times lambda}{4\,pi} \left(\frac{r}{d}\right)^2$$ the Friis attenuation propagation model

Input data set :

E_i	Center (of the AP, the satellite, etc.)	Distance to receiver (samrtphone)
i = 0	(0,5 ; 0,5 ; 0,5)	3
i = 1	(4 ; 0 ; 0)	2
i = 2	(4 ; 5 ; 5)	4,2
i = 3	(3 ; 3 ; 3)	2,5

The estimate position minimize the sum of distances to all spheres.



```java
package nlateration;

import lombok.experimental.FieldDefaults;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

import static java.lang.Math.*;
import static java.lang.System.out;
import static lombok.AccessLevel.PRIVATE;

@FieldDefaults(level = PRIVATE, makeFinal = true)
final class NLateration {

    static byte    X        = 0;
    static byte    Y        = 1;
    static byte    Z        = 2;
    static byte    D        = 3;
    static float[] pChapeau = new float[]{0, 0, 0};

    public static void main(final String[] args) {
        final float[][] tab = new float[4][4];
        tab[0] = new float[]{0.5f, 0.5f, 0.5f, 3};
        tab[1] = new float[]{4, 0, 0, 2};
        tab[2] = new float[]{4, 5, 5, 4.2f};
        tab[3] = new float[]{3, 3, 3, 2.5f};

        double dMin = 0;
        for (final float[] emetteur : tab)
            dMin += abs(emetteur[D] - distance(emetteur[X], emetteur[Y], emetteur[Z]));

        final float[] dMax = new float[3];
        for (int i = 0; i < 3; i++) {
            final List<Float> max = new ArrayList<>();
            for (final float[] t : tab)
                max.add(t[i]);
            dMax[i] = Collections.max(max);
        }
```

```java
        double dMin = 0;
        for (final float[] emetteur : tab)
            dMin += abs(emetteur[D] - distance(emetteur[X], emetteur[Y], emetteur[Z]));

        final float[] dMax = new float[3];
        for (int i = 0; i < 3; i++) {
            final List<Float> max = new ArrayList<>();
            for (final float[] t : tab)
                max.add(t[i]);
            dMax[i] = Collections.max(max);
        }

        final float pas = 0.1f;

        for (float i = pas; i < dMax[X]; i += pas)
            for (float j = pas; j < dMax[Y]; j += pas)
                for (float k = pas; k < dMax[Z]; k += pas) {
                    final double d = distance(i, j, k);
                    if (d < dMin) {
                        dMin = d;
                        p[X] = i;
                        p[Y] = j;
                        p[Z] = k;
                    }
                }

        out.println(Arrays.toString(p));
    }

    private static double distance(final float x, final float y, final float z) {
        return sqrt(pow(x - p[X], 2) + pow(y - p[Y], 2) + pow(z - p[Z], 2));
    }
}
```

```java
import nlateration.Position;

public class GoLateration {

    public static void main(String[] args) {

        Emetteur tab[]  = new Emetteur[4];

        //Déclaration positions Emetteurs

        tab[0] = new Emetteur(new Position(0.5, 0.5, 0.5), 3.0);
        tab[1] = new Emetteur(new Position(4.0, 0.0, 0.0), 2.0);
        tab[2] = new Emetteur(new Position(4.0, 5.0, 5.0), 4.2);
        tab[3] = new Emetteur(new Position(3.0, 3.0, 3.0), 2.5);

        double Dmin; //variable contenant la somme des valeurs des distances
        double D;

        //Position émetteur inconnu
        Position p = new Position(0, 0 , 0);

        //Initilisation du Dmin
        Dmin = Math.abs(Math.sqrt( ( Math.pow(0 - tab[0].getPosition().getX(), 2)  ) +  ( Math.pow( 0 - tab[0].getPosition().getY(), 2) ) +  ( Math.pow( 0 - tab[0].getPosition().getZ(), 2) ) ) - tab[0].getD())
               + Math.abs(Math.sqrt( ( Math.pow( 0 - tab[1].getPosition().getX(), 2)  ) +  ( Math.pow( 0 - tab[1].getPosition().getY(), 2) ) +  ( Math.pow( 0 - tab[1].getPosition().getZ(), 2) ) ) - tab[1].getD())
               + Math.abs(Math.sqrt( ( Math.pow( 0 - tab[2].getPosition().getX(), 2)  ) +  ( Math.pow( 0 - tab[2].getPosition().getY(), 2) ) +  ( Math.pow( 0 - tab[2].getPosition().getZ(), 2) ) ) - tab[2].getD())
               + Math.abs(Math.sqrt( ( Math.pow( 0 - tab[3].getPosition().getX(), 2)  ) +  ( Math.pow( 0 - tab[3].getPosition().getY(), 2) ) +  ( Math.pow( 0 - tab[3].getPosition().getZ(), 2) ) ) - tab[3].getD());

        double dx_max, dy_max, dz_max;
        dx_max = dy_max = dz_max = 10.0;

        double pos = 0.1;

        for (double i = pos; i < dx_max; i += pos) {

            for (double j = pos; j < dy_max; j += pos) {

                for( double k = pos; k < dz_max; k += pos  ) {
                //Pour la position (i, j , k)
                    D = Math.abs(Math.sqrt( ( Math.pow(p.getX() - i, 2)  ) +  ( Math.pow(p.getY() - j, 2) ) +  ( Math.pow(p.getZ() - k, 2) )));
                    if(D < Dmin) {
                        Dmin = D;
                        p.setPosition(i, j , k);
                    }
                }
            }
        }

        System.out.println(Dmin);
    }
}
```

```java
import nlateration.Position;

public class Emetteur {

    private Position position;
    private double d;

    public Emetteur(Position position, double d) {
        this.position = position;
        this.d = d;
    }

    public Position getPosition() {
        return position;
    }

    public double getD() {
        return d;
    }
}
```

TD n°3 Fingerprint

This directed work session consists in building a matrix of an environment. Each case of the environment is characterized with informations :

- Geographic information, for examples :

  - Lattitude, longitude, altitude (universal or global reference system of positioning);
  - The postal address, the stage, the room ;
  - The relative positioning in a 2D plan or 3D plan (local reference system of positioning) ;
- Descriptive information :
  - Signal : Received Signal Strength, Emitted Signal (power) Strength, wave orientation, ratio RSS/ESS => wave propagation model (Friis, …)
  - Quality of communication : received messages over emitted, packet losts, number of damaged messages, …
  - But also

- Needs to catch the actual environment photography of either the MT (Terminal-centric PS) or each component of the infrastruture-centric PS
- Then determine by matching the k best fingerprint closest to the catched environment (above)
  and determine by weighted barycenter computing the positioning of the Object or Human

Below a wrong implementation (I mean with mistakes) of fingerprint implementation.

Consider a rectangle area 12m x 12m of 9 sqaure of 4m x 4m. the left upper side is (0,0) origine. Then each centrum of area can be easily deduced.

In the implementation below, fingerpint initialisation are good, also the RSSI vector of the Mobile Terminal.
But the barycentric ponderation is false.

Determining the k-neighbouring for the k better cells, those cells which have the best minimization of RSSI differences,
that allows to determine the distances d_1, …, d_k.

And knowing d_1 and d_2, it is possible to deduce an expression of coefficient c_2 in terms of c_1.
Such that 0M = c_1 OK_1 + c_2 OK_2 + .. c_k OK_k, with c_1 + … + c_k =1.

if d_1 x alpha = d_2 $\Leftrightarrow$ c_2 = (1/ alpha) c_1

We do the same for c_3 and c_4. Then we can determine the value of c1, then c_2, etc.

And then we can resolve the coordinates of the Mobile Terminal TM.

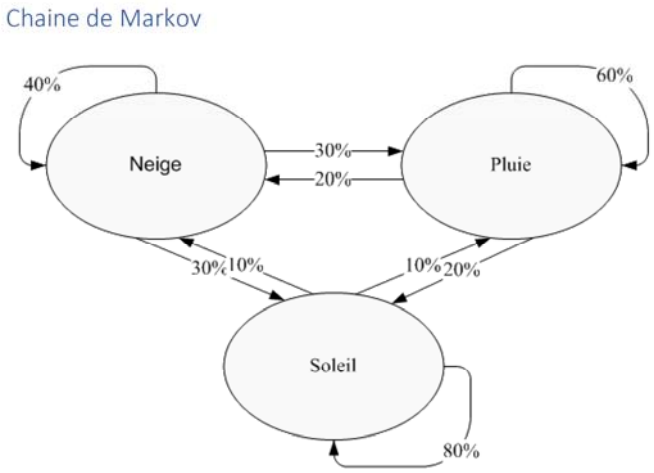
You have to implement it.

```java
import java.lang.reflect.Array;
import java.util.Arrays;

/**
 * Created by vyucel on 29/01/18.
 */
public class GoFingerPrint {

    public static void main(String[] args) {

        Cellule Tf[][] = new Cellule[3][3];

        //Creation des 9 Cellules

        Tf[0][0] = new Cellule(new int[]{-38, -27, -54, -13});
        Tf[0][1] = new Cellule(new int[]{-74, -62, -48, -33});
        Tf[0][2] = new Cellule(new int[]{-13, -28, -12, -40});
        Tf[1][0] = new Cellule(new int[]{-34, -27, -38, -41});
        Tf[1][1] = new Cellule(new int[]{-64, -48, -72, -35});
        Tf[1][2] = new Cellule(new int[]{-45, -37, -20, -15});
        Tf[2][0] = new Cellule(new int[]{-17, -50, -44, -33});
        Tf[2][1] = new Cellule(new int[]{-27, -28, -32, -45});
        Tf[2][2] = new Cellule(new int[]{-30, -20, -60, -40});

        //Création du Terminal Mobile
        Cellule TM = new Cellule(new int[]{-26, -42, -13, -46});

        //Variables

        int kcases[][] = new int[4][2]; // Tableau qui stocke les coordonnées (4) de chaque proche de TM
        boolean presence = false; //Booléen pour vérifie que kcases ne contient pas déjà une certaine case

        //Récupération du max

        int somme = 99999;
        int max[] = new int[2];


        for(int i=0; i < 3; ++i)
        {
            for(int j=0; j < 3; ++j) {

                if(Tf[i][j].somme() < somme) {
                    max[0] = i;
                    max[1] = j;
                }

            }
        }

        //Le max est indispensable pour l'initialisation de chaque kcase[k] (de plus dans notre exemple la case (0,0) est malheureusement la plus petite !

        //recherche des k cases les + proches (et on récupère 2 coordonnées)
        for(int k=0; k < 4; k++) {
```

```
            }

        }
    }

    //Le max est indispensable pour l'initialisation de chaque kcase[k] (de plus dans notre exemple la case (0,0) est malheureusement la plus petite !

    //recherche des k cases les + proches (et on récupère 2 coordonnées)
    for(int k=0; k < 4; k++) {

        kcases[k][0] = max[0];
        kcases[k][1] = max[1];
        //Math.abs(Tf[0][0].somme() - TM.somme());

        for (int i = 0; i < 3; ++i) {
            for (int j = 0; j < 3; ++j) {

                //On vérifie que le couple i,j n'est pas déjà contenu dans les kcases précedentes
                for (int l=0; l < k; ++l) {
                    if(kcases[l][0] == i && kcases[l][1] == j)
                        presence =true;

                }

                ///Si le couple n'est pas présent
                if(!presence) {
                    if (Math.abs(Tf[i][j].somme() - TM.somme()) < Math.abs(Tf[kcases[k][0]][kcases[k][1]].somme() - TM.somme()) ) {
                        kcases[k][0] = i;
                        kcases[k][1] = j;
                    }
                }
                //Réinitialisation de k
                presence = false;
            }
        }

    }

    int moy_x, moy_y;
    moy_x = moy_y = 0;
    //On affiche les coordonnées du centre de chaque case (i et j *4 + 2) et on affecte les nouvelles coordonnées pour le calcul
    for (int element[]: kcases) {
        element[0] = element[0]*4 +2;
        element[1] = element[1]*4 + 2;

        moy_x += element[1];
        moy_y += element[0];

        //Au vue de la manière dont sont stockés les tableaux on doit inverser les i et j
        System.out.println("K proches : " + element[1] + ", " + element[0]);
    }

    System.out.println("Centre gravitoire: " + moy_x/4 + ", " + moy_y/4);
  }

}
```

```
/**
 * Created by vyucel on 29/01/18.
 */
public class Cellule {

    int[] tpr; //tableau de 4 puissances

    public Cellule(int[] tpr) {
        this.tpr = tpr;
    }

    public int somme() {
        int somme=0;

        for(int i=0; i< tpr.length; ++i) {
            somme += tpr[i];
        }

        return somme;
    }
}
```

TD n°1 Markov Model usefull for anonymisation and for prediction (positioning and action)



What is Markov Model or Hidden Markov Model ?
An interesting computer science object interesting to :

- modelize an automata,
- a statistical and valued graph

which allows, firstly to summarize actions performed (the history of), and secondly which predicts the next position or action, or/and the previous position or action.

MM is composed of nodes and transitions.

Localization Graph or Positioning graph

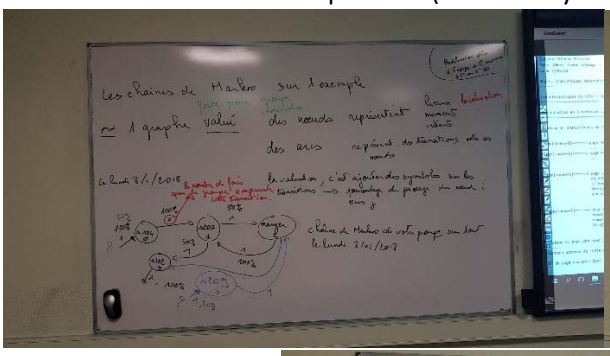
<display an example in a room, extended then to a building, finally extended to a city>

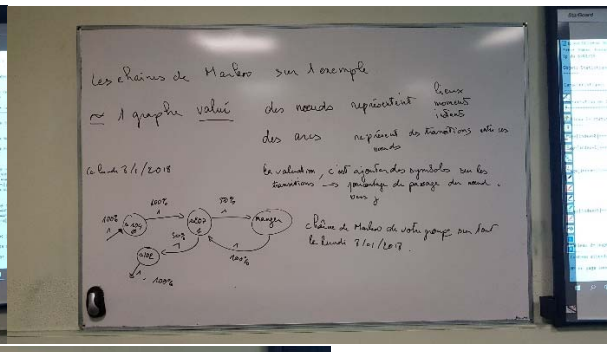
Practical work

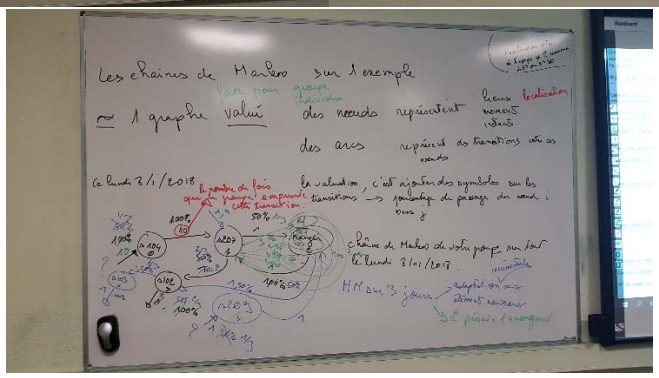
Object: observed statistics and predictions of a web site access (from the same site)

In the sequel, we detail the
Web site characteristics : it is composed of 5 pages

See below pictures (french no yet transduced) of this directed work.







Board 1= statistics of transitions pages/states

Page[index=0]======> page numéro index +1
...
Page[index=1]======> page numéro index +1
...
...
...
Page[index=i]======> page numéro index +1
nb_hit // nombre de hits pour accéder à cette page
// nombre de transitions depuis cette page
Vers[j variant de 0..5] contenant le nombre de hit de i vers j
sum_nb_of_hit // Sum_j=0^5 vers[j]
...
...
Page[index=5]======> page numéro 6
nombre de hits = nb_hit // pour accéder à cette page
nombre de transitions depuis cette page
Vers[j variant de 0..5] contenant le nombre de hit de la page 6 (=i+1) vers j

board 2= page.php avec [src=optionnel] et dst=obligatoire
Contenu attendu de cette page dynamique.

N° de page courante $dst
provenant de $src

choix du prochain hit (de la prochaine page):
page 1 ...

page 2 ...
...
page j avec page.php et src=i et dst=j
...
page i avec page.php et src=i et dst=i
...
page 5 ...

Statistiques actuelles :
+ le tableau brut
+ et/ou le modèle de markov version txt

depuis page 1
.....
.....
depuis page l
Nombre de transitions pour arriver sur l
nbhit=page[$l].nb_hit
Nombre de transitions effectuées depuis $l
page[$l].sum_nb_of_hit
transition \ Nombre | pourcentage
Vers 1 page[l].vers[0] / page[l].vers[0]/page[l].sum_nb_of_hit
...
Vers p page[l].vers[p-1] / page[l].vers[p-1]/page[l].sum_nb_of_hit
...
vers 6 page[l].vers[5] / page[l].vers[5]/page[l].sum_nb_of_hit

<u>board 3</u>
Scénario
page 1 nbhit+=1
vers page1 nbhi+=1 => 2
nbhit depuis 1 versqqch +=1
nbhit de 1 vers 1 +=1
Vers page 6 ...

Représentation du modèle de markov
Etat
1, 2hit ---> 1
1/3, 33%

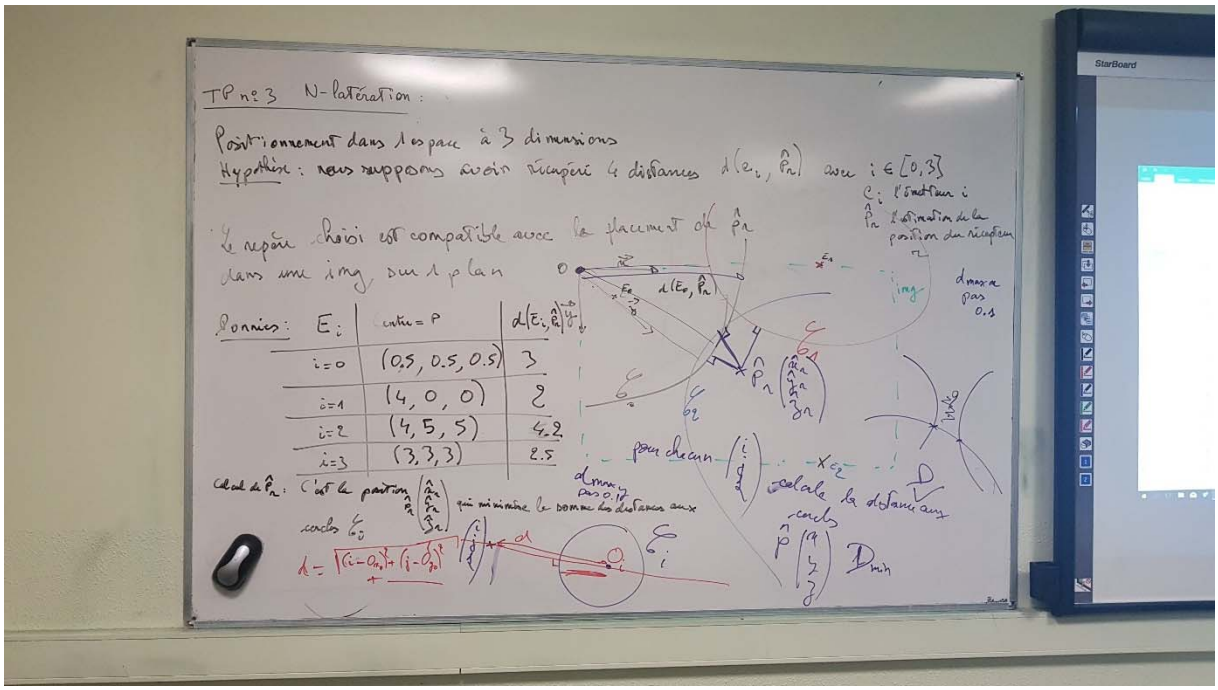
---> 6
2/3, 66%

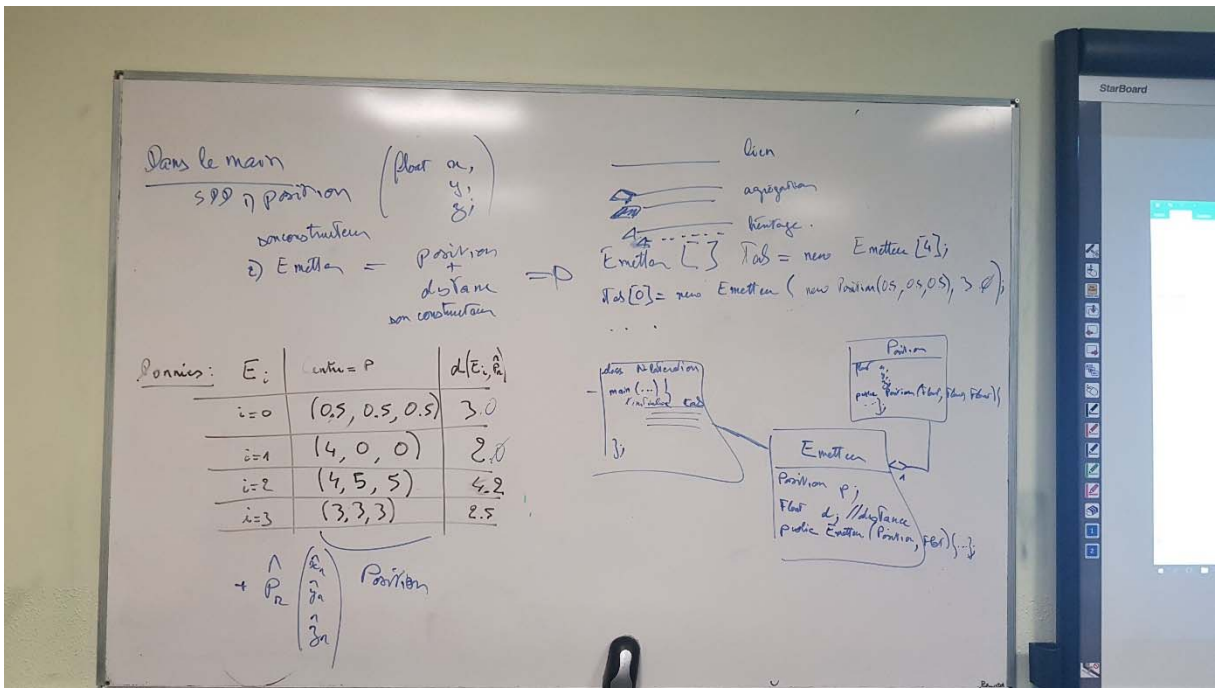
3, 1hit ---> 1
1/1, 100 %

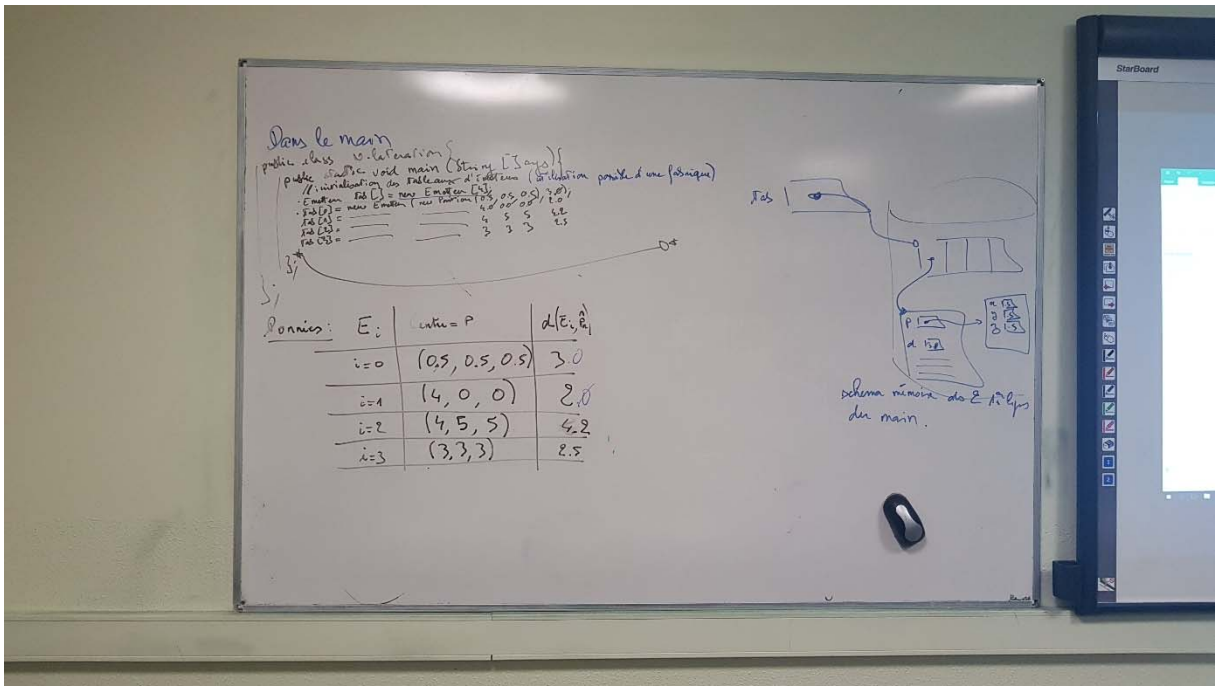
6, 1hit

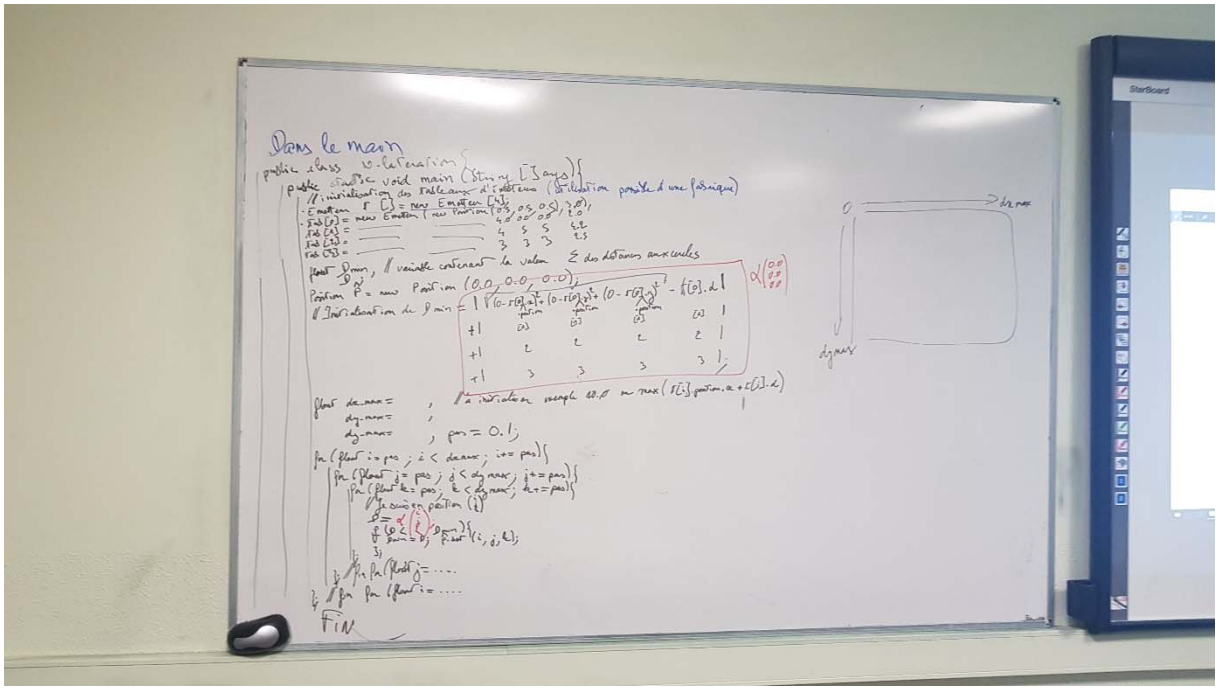
Annexe 1 Resolution of N-lateration exercise

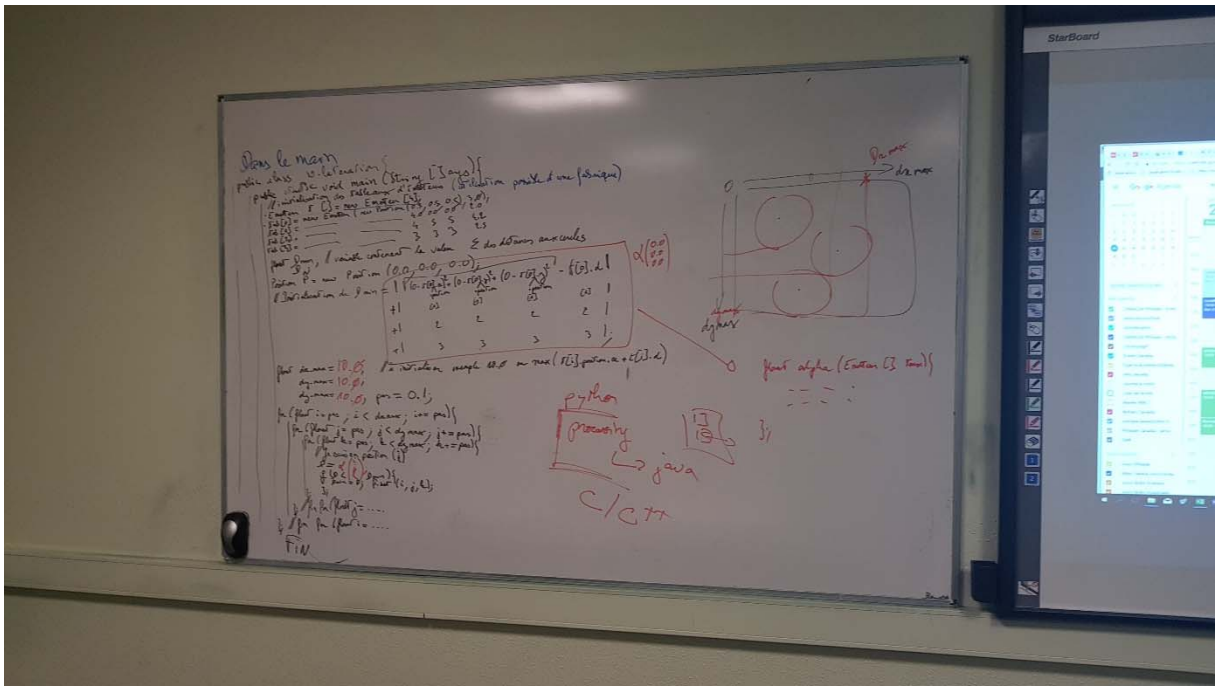
See below, 5 pictures which illustrate an uncorrect but helpfull resolution tentative of this problem.






Dans le main
public class O_Laterarium {
public static void main (String [] args){
// initialisation des tableaux d'entiers (solution possible d'une fabrique)
Emetteur [] = new Emetteur [3];
Tab [0] = new Emetteur (new Position (0.5, 0.5, 0.5), 3.0, 1.0
Tab [1] = 4 5 5 4.2 1.5
Tab [2] = 3 3 3
Tab [3] =
Données :
Ei
centre = P
d(Ei, P)
i=0
(0,5, 0.5, 0.5)
3.0
i=1
(4, 0, 0)
2.0
i=2
(4, 5, 5)
4.2
i=3
(3, 3, 3)
2.5
Tab
P
d
schema mémoire de E, P, d pdt le main.


Dans le main
public class O_Laterarium {
public static void main (String [] args){
// initialisation des tableaux d'entiers (solution possible d'une fabrique)
Emetteur [] = new Emetteur [3];
Tab [0] = new Emetteur (new Position (0.5, 0.5, 0.5), 3.0, 1.0
Tab [1] = 4 5 5 4.2 1.5
Tab [2] = 3 3 3
Tab [3] =
float dmin, // variable contenant la valeur E des distances aux cercles
Position P = new Position (0.0, 0.0, 0.0);
// initialisation de dmin =
+1
+1
+1
float de_max = ,
dy_max = ,
dy_max = , pas = 0.1;
for (float i = pas ; i < de_max ; i += pas){
for (float j = pas ; j < dy_max ; j += pas){
for (float k = pas ; k < dy_max ; k += pas){
// la case en position (i,j,k)
d =
if (d < dmin) dmin = d;
}
}
}
FIN
0
de max
dy max


StarBoard
Dans le main
python
processing
java
C/C++
FIN

Annexe 2 Resolution of Markov Model Prediction exercise

See below, 4 pictures which illustrate an uncorrect but helpfull resolution tentative of this problem.

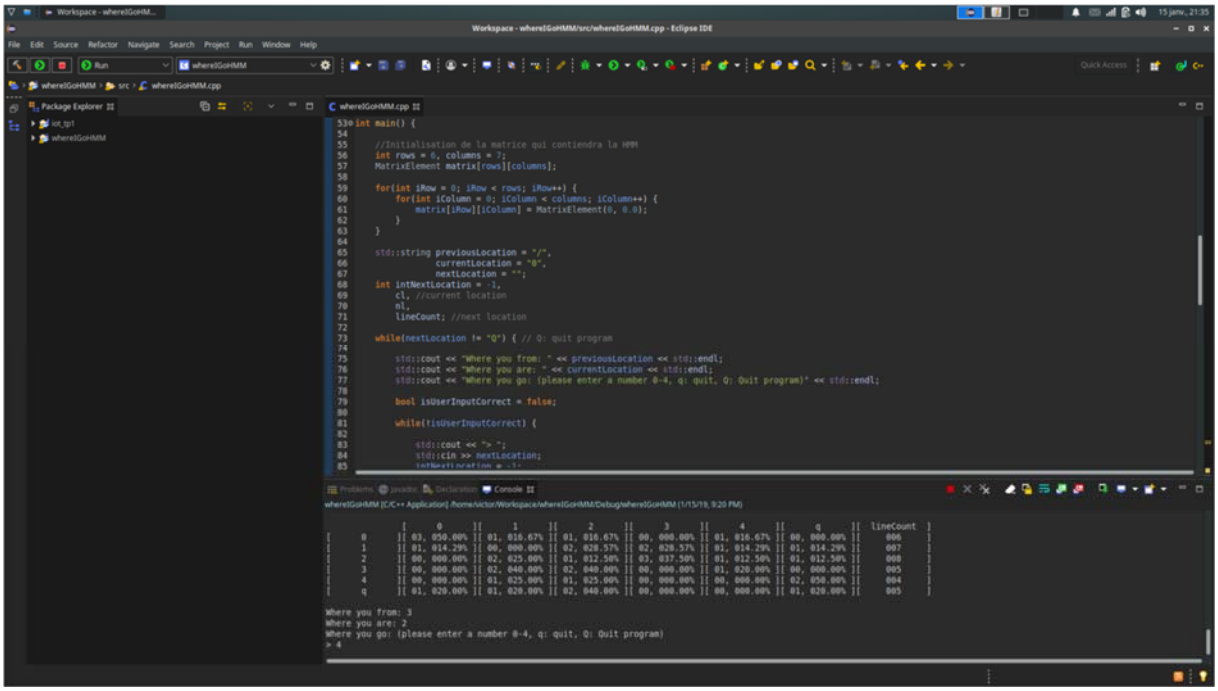
```java
/**
 * @author: Vincent Yucel
 * @version : 1
 *
 */
package markov;

import java.util.Scanner;

public class InitMarkov {


    public static void main(String[] args) {
        //Variables

        final int nbL = 5, nbC = 6;
        int max_cellule = 0;
        Cellule MM[][] = new Cellule[nbL][nbC];

        //affichage, réinit

        int prev = -1, curr = 0;
        int nextPage = -1;

        for (int i = 0; i < nbL; ++i) {

            for (int j = 0; j < nbC; ++j) {

                MM[i][j] = new Cellule(0, 0);

            }
        }

        while (true) {
            //affichage
            System.out.println("Vous venez de " + prev + "." + "\nVous êtes en " + curr);

            //lecture de la prochaine page
            Scanner sc = new Scanner(System.in);
            System.out.println("Prochaine page (sortie du programme " + nbL + ") : ");
            nextPage = sc.nextInt();

            while (nextPage < 0 || nextPage >= nbL) {

                if (nextPage == nbL) {
                    System.out.println("Sortie du programme");
                    System.exit(1);
                }

                System.out.println("Prochaine page (sortie du programme " + nbL + ") : ");
                nextPage = sc.nextInt();

            }
            prev = curr;
            curr = nextPage;
```

```
            /*MM[curr][prev].nb += 1;
        MM[curr][nbC - 1].nb += 1;*/
            MM[prev][curr].nb += 1;
            MM[prev][nbC - 1].nb += 1;

            for (int k = 0; k < nbC; ++k) {
                MM[prev][k].stat = (float) (MM[prev][k].nb) / (float) (MM[prev][nbC - 1].nb);
            }

            afficheTab(MM, nbC, nbL);
            System.out.print("\n");
        }

    }

    public static void afficheSeparateur(int n, int deb_ligne) {
        int i;

        //Tabulation avant chaque ligne pour les titres
        for(i =0; i < deb_ligne; i++)
            System.out.print(" ");

        for (i = 0; i < n; i++)
            System.out.print("+----------------------------------");
        System.out.print("+");
    }

    public static void afficheTab(Cellule tab[][], int col, int ligne) {
        int i, j;
        String sep = "+----------------------------------";
        String prev = "Page précédente : 0";

        //Affichage tabulation première ligne avant titre colonnes
        for(int k = 0; k < prev.length(); k++)
            System.out.print(" ");


        //Affichage des titres
        for (i = 0; i < col - 1; i++)
        {
            String titre = "Page courante : " + i;
            System.out.print(titre);
            for(int k = 0; k < sep.length() - titre.length(); k++)
                System.out.print(" ");
        }
        System.out.print("Totaux :");
        System.out.print("\n");

        //-------------------------------------------------

        //Affichage des lignes et colonnes

        for (i = 0; i < ligne; i++) {

            prev = "Page précédente : " + i;

            afficheSeparateur(col, prev.length());
```

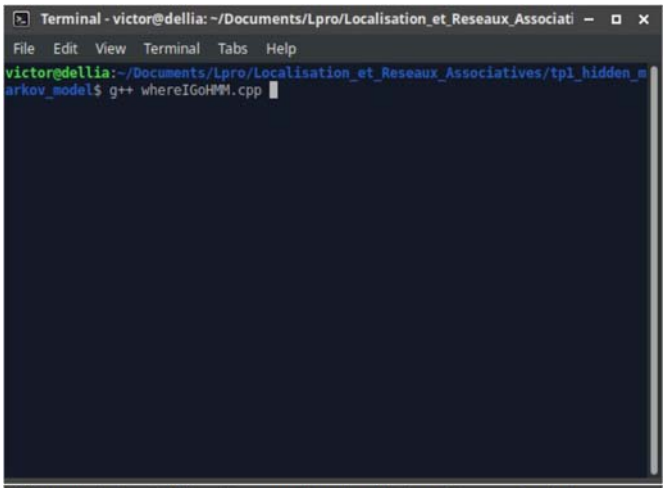
```
            afficheSeparateur(col, prev.length());
            System.out.print("\n");

            for (j = 0; j < col; j++) {

                if(j == 0)
                    System.out.print(prev);

                String contenu = "|Nombre : " + tab[i][j].nb + " , Stat : " + tab[i][j].stat*100 + "%";
                System.out.print(contenu);

                for(int k = 0; k < sep.length() - contenu.length(); k++)
                    System.out.print(" ");

            }

            System.out.print("|");
            System.out.print("\n");

        }

        afficheSeparateur(col, prev.length());
    }

}
```

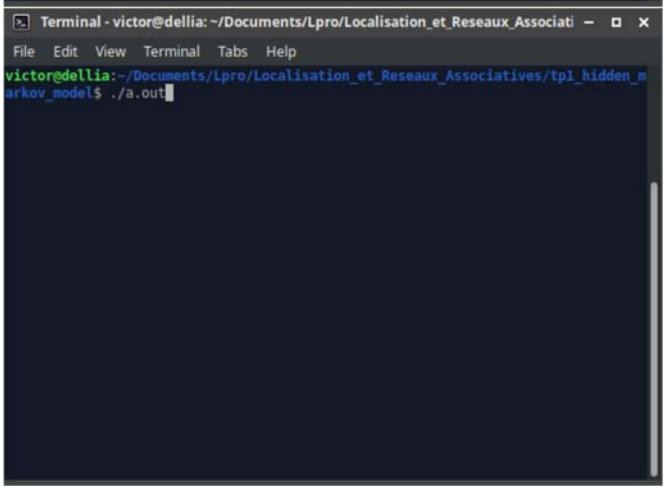
```
/**
 * @author: Vincent Yucel
 * @version : 1
 *
 */

package markov;

public class Cellule {

    public int nb;
    public float stat;

    public Cellule(int _nb, float _stat) {

        this.nb = _nb;
        this.stat = _stat;
    }
}
```

Another example of resolution :



To compile and execute :





Exemple of execution :

Execution:

Exemple d’exécution suivant ce déroulement

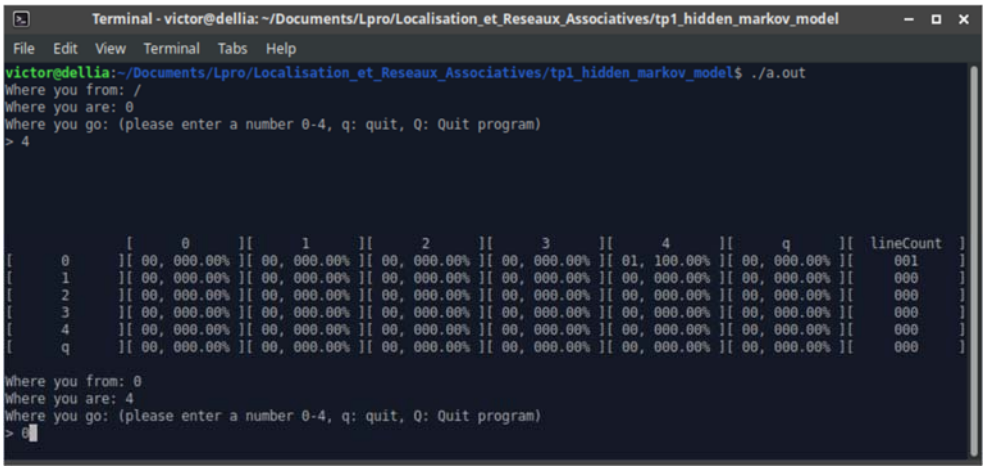
0→4

0→4→0

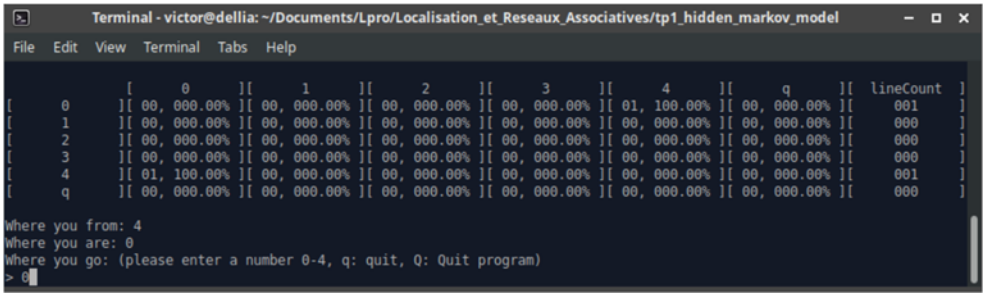
0→4→0→0

0→4→0→0→4

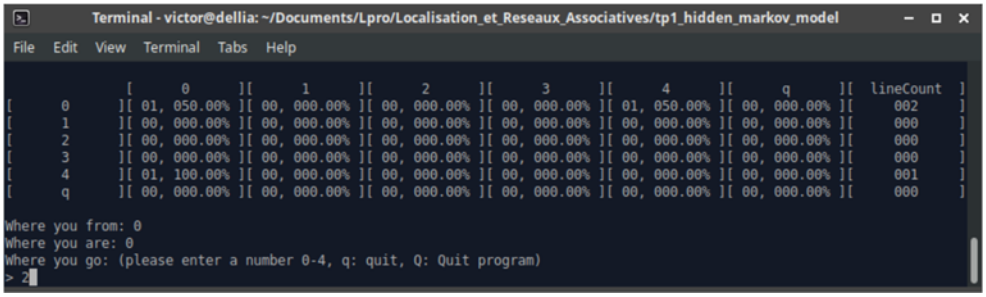
0→4:



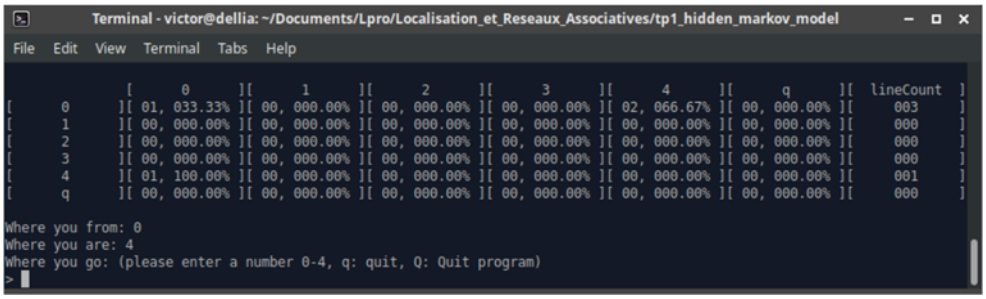
4→0:



0→0:



0→4



Another, and last exemple of execution :